STM32 Based Project: UART

MANUAL

SYED SHIFAT

Table of Contents

Project Features 2

- UART Communication 2
- Timer-Based Data Sending 2
- Photoresistor Reading and Threshold Detection (ADC + UART) 2
- External Interrupt Handling 2

STM32 Features Used in the Project 2

Project functions 2

Circuit diagram: 3

Pin configuration: 3

Function description 4

- 1. Function for Clock Enable 4
- 2. Function for Pin Config 4
- 3. Function for SysTick Setup 5
- 4. Function for Time Delay 5
- 5. Function for UART Config 6
- 6. Function for ADC Configuration 6
- 7. Timer Config Function 7
- 8. External Interrupt configuration function 7
- 9. All Interrupt Vector Function 8
  - USART1_IRQHandler 8
  - TIM2_IRQHandler 9
  - ADC1_2_IRQHandler 10
  - EXTI2_IRQHandler 10
- 9. Main Function 11

PCB design 11

Test Results: 12

- UART Communication Test 12
- Timer Functionality 12
- ADC (Photoresistor) Test 13
- External Interrupt Test (PA2) 13

Conclusion 13

STM32 Based Project

This project involves communication between two STM32 microcontrollers using UART. When data is sent, the sending STM32 blinks an LED on pin PA0, and when data is received, it blinks an LED on PA1. A timer is used to send numbers from 0 to 3 every 3 seconds. On the receiving STM32, pins PA4 and PA7 are used to show the binary form of the received number, with PA4 showing the least significant bit (LSB) and PA7 showing the most significant bit (MSB). The system also reads values from a photoresistor using ADC. If the light level goes above a set high value, PA5 turns ON. If it drops below a low value, PA5 turns OFF on the other STM32. Additionally, when an external interrupt happens on pin PA2, a signal is sent to the second STM32, which then toggles an LED on PA6.

Github link: https://github.com/shifat65/3200_project_STM32_UART_ADC.git

Project Features

UART Communication

The STM32 communicates with another STM32 using UART.

- When data is sent, PA0 blinks.
- When data is received, PA1 blinks.

Timer-Based Data Sending

A general-purpose timer sends data (from 0 to 3 in sequence) at regular intervals (3s). PA4 and PA7 act as indicators for the received data:

- PA4 shows the LSB (Least Significant Bit)
- PA7 shows the MSB (Most Significant Bit)

Photoresistor Reading and Threshold Detection (ADC + UART)

The system reads values from a photoresistor.

- If the value goes above a high threshold, PA5 turns ON.
- If the value goes below a low threshold, PA5 turns OFF on the other STM32.

External Interrupt Handling

When an external interrupt occurs on PA2, a signal is sent.

- The other STM32 responds by toggling PA6.

STM32 Features Used in the Project

- UART
- ADC
- SysTick timer
- Timer GPIO trigger
- External interrupt
- All handled via interrupts

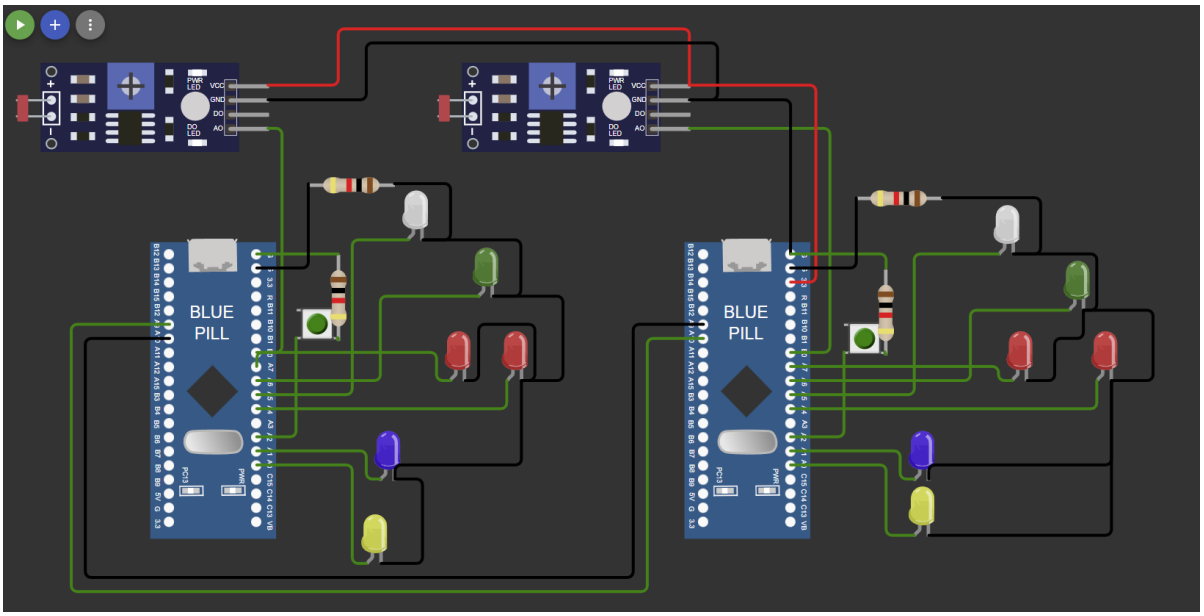
Project functions

1. **Function for clock**
    - void En_clock(void)
2. **Function for pin config**

- void gpio_setup(void)

3. **Function for systick setup**
   - void systick_config(void)
4. **Function for time delay**
   - void delay_ms(void)
   - void delay(uint32_t count)
5. **Function for UART config**
   - void Uart1_config(void)
6. **Function for ADC config**
   - void ADC_config(void)
7. **Timer config function**
   - void timer_config(void)
8. **All interrupt vector functions**
   - void TIM2_IRQHandler(void)
   - void USART1_IRQHandler(void)
   - void ADC1_2_IRQHandler(void)
   - void EXTI2_IRQHandler(void)
9. **Main function**
   - int main(void)

Circuit diagram:



Pin configuration:

Pin Name	Purpose	Config	CNF[1:0] + MODE[1:0]
PA0	TX indication (blink on send)	GPIO Output Push-Pull	0011
PA1	RX indication (blink on receive)	GPIO Output Push-Pull	0011
PA2	External interrupt input	GPIO Input with Pull-up/down	1000

PA4	Display LSB of data sent by Timer interrupt	GPIO Output Push-Pull	0011
PA5	Output control (based on photoresistor)	GPIO Output Push-Pull	0011
PA6	Toggle on external interrupt response	GPIO Output Push-Pull	0011
PA7	Display MSB of data sent by Timer interrupt	GPIO Output Push-Pull	0011
PA9	UART Transmission (USART1_TX)	Alternate Function Push-Pull	1011
PA10	UART Reception (USART1_RX)	Input Floating	0100
PB0	Photoresistor input (ADC)	Analog Input	0000

Function description

1. Function for Clock Enable

void En_clock(void)

- Enables peripheral clocks:
  - APB1ENR: USART2, TIM2
  - APB2ENR: AFIO, IOPA, IOPB, USART1, ADC1

```
void En_clock(void)
{
    RCC->APB1ENR |= RCC_APB1ENR_USART2EN
                  | RCC_APB1ENR_TIM2EN;

    RCC->APB2ENR |= RCC_APB2ENR_AFIOEN
                  | RCC_APB2ENR_IOPAEN
                  | RCC_APB2ENR_IOPBEN
                  | RCC_APB2ENR_USART1EN
                  | RCC_APB2ENR_ADC1EN
                  | RCC_APB2ENR_ADC1EN;
}
```

2. Function for Pin Config

void gpio_setup(void)

- Configures GPIO pins:
  - PA0–PA7: Various input/output configurations for UART, ADC, EXT, response signals
  - PA9: UART1 Tx → AF Output Push-Pull (1011)
  - PA10: UART1 Rx → Input floating (0100)
  - PB0: Analog input (ADC) → 0000

```
void gpio_setup(void)
{
    // PA0 Tx signal for uart1:GPIO output pushpull 0011
    GPIOA->CRL &= ~(GPIO_CRL_CNF0 | GPIO_CRL_MODE0);
    GPIOA->CRL |= GPIO_CRL_MODE0;
    // PA1 Rx signal for uart1:GPIO output pushpull 0011
```

```
    GPIOA->CRL &= ~(GPIO_CRL_CNF1 | GPIO_CRL_MODE1);
    GPIOA->CRL |= GPIO_CRL_MODE1;
    // PA2 user input 1
    GPIOA->CRL &= ~(GPIO_CRL_CNF2 | GPIO_CRL_MODE2);
    GPIOA->CRL |= GPIO_CRL_CNF2_0;
    // PA3 user input 0;
    GPIOA->CRL &= ~(GPIO_CRL_CNF3 | GPIO_CRL_MODE3);
    GPIOA->CRL |= GPIO_CRL_CNF3_0;
    // PA4 Output for USART1 receive use 0011
    GPIOA->CRL &= ~(GPIO_CRL_CNF4 | GPIO_CRL_MODE4);
    GPIOA->CRL |= GPIO_CRL_MODE4;
    // PA5 ADC response (other stm) push pull 0011
    GPIOA->CRL &= ~(GPIO_CRL_CNF5 | GPIO_CRL_MODE5);
    GPIOA->CRL |= GPIO_CRL_MODE5;
    // PA6 EXT interrupt response GPIO output push pull 0011
    GPIOA->CRL &= ~(GPIO_CRL_CNF6 | GPIO_CRL_MODE6);
    GPIOA->CRL |= GPIO_CRL_MODE6;
    // PA7 Output for USART1 receive use 0011
    GPIOA->CRL &= ~(GPIO_CRL_CNF7 | GPIO_CRL_MODE7);
    GPIOA->CRL |= GPIO_CRL_MODE7;
    // PA9 UART1 Tx : Output AF push-pull 1011
    GPIOA->CRH &= ~(GPIO_CRH_CNF9 | GPIO_CRH_MODE9);
    GPIOA->CRH |= GPIO_CRH_CNF9_1 | GPIO_CRH_MODE9;
    // PA10 UART1 RX : input floating 0100
    GPIOA->CRH &= ~(GPIO_CRH_CNF10 | GPIO_CRH_MODE10);
    GPIOA->CRH |= GPIO_CRH_CNF10_0;
    // PB0 analog input for ADC 0000
    GPIOB->CRL &= ~(GPIO_CRL_CNF0 | GPIO_CRL_MODE0);
}
```

3. Function for SysTick Setup

void systick_config(void)

- SysTick LOAD = 72000-1 (for 1 ms delay at 72 MHz)
- Clock source: CPU
- Enables SysTick

```
void systick_config(void)
{
    SysTick->LOAD = 72000 - 1;
    SysTick->VAL = 0;
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE
                    | SysTick_CTRL_ENABLE;
}
```

4. Function for Time Delay

void delay_ms(void) and void delay(uint32_t count)

- delay_ms() waits for the COUNTFLAG of SysTick
- delay(count) repeats delay_ms() for count times

```
void delay_ms(void)
```

```
{
    while (!(SysTick->CTRL & SysTick_CTRL_COUNTFLAG))
    {
    }
}

void delay(uint32_t count)
{

    while (count--)
    {
        delay_ms();
    }
}
```

5. Function for UART Config

void Uart1_config(void)

- CR1:
  - TE (bit 3): Transmitter enable
  - RE (bit 2): Receiver enable
  - RXNEIE (bit 5): Rx not empty interrupt enable
  - UE (bit 13): USART enable
- Baud Rate: USART1->BRR = 0x1DCC (9600 for 72 MHz)
- GPIO Config: PA9 (Tx), PA10 (Rx)
- NVIC interrupt enable: USART1_IRQn
- Transmit: Write to USART1->DR
- Receive: USART1_IRQHandler handles reception using RXNE flag

```
void Uart1_config(void)
{
    USART1->CR1 |= USART_CR1_TE
                 | USART_CR1_RE
                 | USART_CR1_RXNEIE
                 | USART_CR1_UE;

    // Baud rate is 9600;
    USART1->BRR = 0x1DCC;

    NVIC_EnableIRQ(USART1_IRQn);
}
```

6. Function for ADC Configuration

void ADC_config(void)

- CR1:
  - AWDEN (bit 23): Analog Watchdog enable
  - AWDIE (bit 6): Analog Watchdog interrupt enable
  - AWDCH = 8 (B0 = Channel 8 → 01000)
- CR2:
  - CONT (bit 1): Continuous conversion
  - ADON (bit 0): ADC enable (called twice as per manual)

- SQR3: Set channel 8 (B0)
- HTR/LTR: High/Low threshold for watchdog
- Interrupt: ADC1_2_IRQHandler
    - SR.AWD checked
    - Action based on ADC1->DR compared to thresholds
    - AWD flag cleared after handling

```
void ADC_config(void)
{
    ADC1->CR1 |= ADC_CR1_AWDEN | ADC_CR1_AWDIE | ADC_CR1_AWDCH_3;
    // B0 analog input pin
    ADC1->SQR3 = ADC_SQR3_SQ1_3;
    ADC1->CR2 |= ADC_CR2_CONT;

    ADC1->CR2 |= ADC_CR2_ADON;

    ADC1->HTR = 0xA00;
    ADC1->LTR = 0x600;

    delay(500);
    ADC1->CR2 |= ADC_CR2_ADON;

    NVIC_EnableIRQ(ADC1_2_IRQn);
}
```

7. Timer Config Function

void timer_config(void)

- CNT = 0
- PSC = 7200 - 1 (CK_CNT = 72MHz / 7200 = 10KHz)
- ARR = 50000 (5s period)
- DIER bit 0: Enable update interrupt
- CR1 bit 4: Downcount (DIR)
- CR1 bit 0: Timer Enable
- NVIC: TIM2_IRQn enabled
- ISR: Toggle PA0, send UART1 Sdata and increment cyclically

```
void timer_config(void)
{
    TIM2->CNT = 0;
    TIM2->PSC = 7200 - 1;
    TIM2->ARR = 50000;
    TIM2->DIER |= TIM_DIER_UIE;
    TIM2->CR1 |= TIM_CR1_DIR;
    TIM2->CR1 |= TIM_CR1_CEN;

    NVIC_EnableIRQ(TIM2_IRQn);
}
```

8. External Interrupt configuration function

- AFIO->EXTICR[1] = 0; Selecting PA as External Interrupt pin
- IMR: Unmasking.

- RTSR: Interrupt on Raising.

```
void Exi_config(void)
{
    AFIO->EXTICR[1] = 0;

    EXTI->IMR |= EXTI_IMR_MR2;
    EXTI->RTSR |= EXTI_RTSR_TR2;

    NVIC_EnableIRQ(EXTI2_IRQn);
    // NVIC_EnableIRQ(EXTI3_IRQn);
}
```

9. All Interrupt Vector Function

USART1_IRQHandler

- Check RXNE
- Read from USART1->DR
- Conditional actions using GPIO depending on Rdata
- Clear RXNE flag
- Delay and reset output pin

```
void USART1_IRQHandler(void)
{

    if (USART1->SR & USART_SR_RXNE)
    {
        Rdata = USART1->DR;
        USART1->SR &= ~USART_SR_RXNE;
        GPIOA->ODR |= GPIO_ODR_ODR1;

        if (Rdata == 0)
        {
            GPIOA->ODR &= ~GPIO_ODR_ODR4;
            GPIOA->ODR &= ~GPIO_ODR_ODR7;
        }
        else if (Rdata == 1)
        {
            GPIOA->ODR |= GPIO_ODR_ODR4;
            GPIOA->ODR &= ~GPIO_ODR_ODR7;
        }
        else if (Rdata == 2)
        {
            GPIOA->ODR &= ~GPIO_ODR_ODR4;
            GPIOA->ODR |= GPIO_ODR_ODR7;
        }
        else if (Rdata == 3)
        {
            GPIOA->ODR |= GPIO_ODR_ODR4;
```

```
            GPIOA->ODR |= GPIO_ODR_ODR7;
        }

        if (Rdata == 'A')
        {
            GPIOA->ODR ^= GPIO_ODR_ODR6;
        }

        if (Rdata == 'B')
        {
            GPIOA->ODR |= GPIO_ODR_ODR5;
        }
        else if (Rdata == 'C')
        {
            GPIOA->ODR ^= GPIO_ODR_ODR5;
        }

        delay(25);
        GPIOA->ODR &= ~GPIO_ODR_ODR1;
    }
}
```

-

TIM2_IRQHandler

- Check UIF, clear it
- Toggle PA0
- Send Sdata via UART1
- Cycle Sdata from 0 to 3

```
void TIM2_IRQHandler(void)
{
    if (TIM2->SR & TIM_SR_UIF)
    {
        TIM2->SR &= ~TIM_SR_UIF;

        GPIOA->ODR |= GPIO_ODR_ODR0;

        USART1->DR = Sdata;

        delay(25);
        GPIOA->ODR &= ~GPIO_ODR_ODR0;

        Sdata++;
        if (Sdata == 4)
        {
            Sdata = 0;
        }
    }
}
```

ADC1_2_IRQHandler

- Read ADC1->DR
- If AWD:
    - Compare with HTR and LTR
    - Set present flag
    - If changed from past, send 'B' or 'C' via UART1
- Clear AWD flag at the end

```
void ADC1_2_IRQHandler(void)
{
    Adc_data = ADC1->DR;

    if (ADC1->SR & ADC_SR_AWD)
    {
        if (Adc_data > ADC1->HTR)
        {
            present = 1;
        }
        else if (Adc_data < ADC1->LTR)
        {
            present = 0;
        }

        if ((present == 1) && (present != past))
        {
            GPIOA->ODR |= GPIO_ODR_ODR0;
            USART1->DR = 'B';
            past = present;
            delay(25);
            GPIOA->ODR &= ~GPIO_ODR_ODR0;
        }
        else if ((present == 0) && (present != past))
        {
            GPIOA->ODR |= GPIO_ODR_ODR0;
            USART1->DR = 'C';
            past = present;
            delay(25);
            GPIOA->ODR &= ~GPIO_ODR_ODR0;
        }

        ADC1->SR &= ~ADC_SR_AWD;
    }}
```

EXTI2_IRQHandler

- Clear pending bit
- Toggle PA0
- Send 'A' via UART1

```
void EXTI2_IRQHandler(void)
{
    if (EXTI->PR & EXTI_PR_PR2)
    {
        EXTI->PR |= EXTI_PR_PR2;
    }
    GPIOA->ODR |= GPIO_ODR_ODR0;
    USART1->DR = 'A';
    delay(25);
    GPIOA->ODR &= ~GPIO_ODR_ODR0;
}
```

9. Main Function

int main(void)

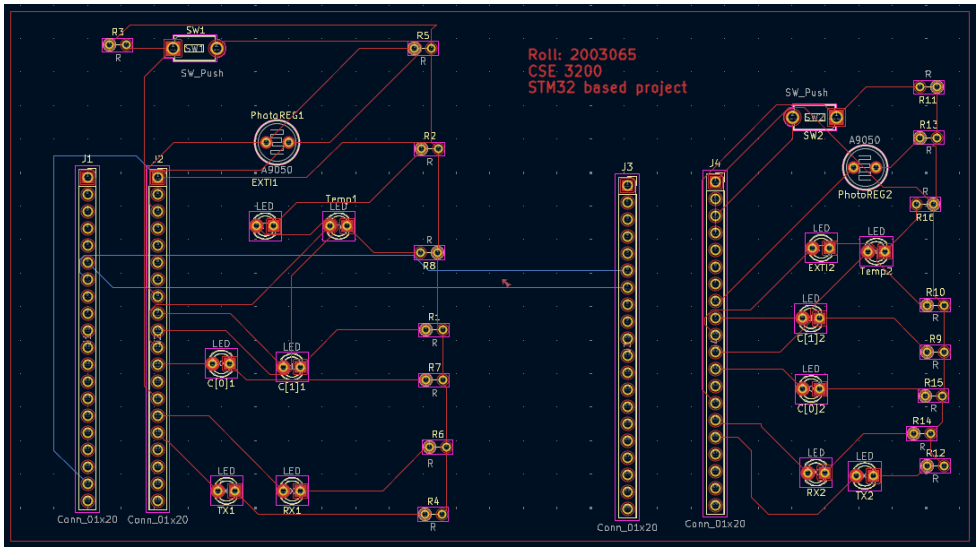
- Calls:
  - En_clock()
  - gpio_setup()
  - systick_config()
  - timer_config()
  - Uart1_config()
  - Exi_config()
  - ADC_config()
- Infinite loop: waits for interrupts to perform tasks

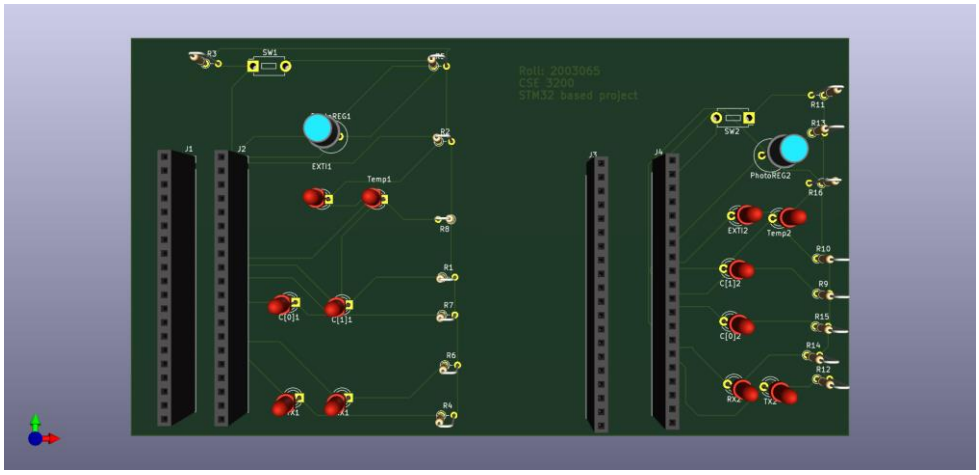
```
int main(void)
{
    En_clock();
    gpio_setup();
    systick_config();
    timer_config();
    Uart1_config();
    Exi_config();
    ADC_config();

    while (1)
    {
    }
    return 0;
}
```

PCB design

PCB folder contains the files of PCB design. PCB is designed using KiCad.





Test Results:

UART Communication Test

- **Sending (PA0):**
  - Every 5 seconds, a value from 0 to 3 is sent using USART1.
  - PA0 blinks shortly when data is sent (as visual indication).
- **Receiving (PA1):**
  - Data is received via USART1.
  - PA1 blinks shortly upon receiving any data.
- **Data Decoding on Receiver STM32:**
  - Data (0 to 3) sets **PA4 (LSB)** and **PA7 (MSB)** accordingly:
    - 0 → 00: PA4=LOW, PA7=LOW
    - 1 → 01: PA4=HIGH, PA7=LOW
    - 2 → 10: PA4=LOW, PA7=HIGH
    - 3 → 11: PA4=HIGH, PA7=HIGH

Timer Functionality

- **Timer2** triggers an interrupt every 3 seconds.
  - In interrupt:
    - A new value (0–3) is sent over UART.

- **PA0 blinks** with each send event.
- Value loops back to 0 after reaching 3.

ADC (Photoresistor) Test

- Reads analog voltage on **PB0**.
- ADC Watchdog compares input with:
  - **High Threshold (HTR):** 0xA00 (~2560)
  - **Low Threshold (LTR):** 0x600 (~1536)
- **If light increases (ADC > HTR):**
  - Flag present = 1
  - Sends 'B' over UART
  - **PA0 blinks** (send indicator)
  - **Receiver turns ON PA5**
- **If light decreases (ADC < LTR):**
  - Flag present = 0
  - Sends 'C' over UART
  - **PA0 blinks**
  - **Receiver toggles or turns OFF PA5**

External Interrupt Test (PA2)

- On rising edge at **PA2**:
  - External interrupt EXTI2_IRQHandler() is called.
  - Sends 'A' via UART.
  - **PA0 blinks** to indicate send.
  - Receiver toggles **PA6** in response.

Feature	Expected Result	Test Status
UART Send/Receive	Data 0–3 sent every 3 sec, PA0/PA1 blink	Working
LED Pattern (PA4, PA7)	Matches received data in binary	Working
ADC Threshold Response	PA5 ON/OFF/toggle depending on light	Working
External Interrupt	Toggle PA6 on remote board on PA2 trigger	Working
Timer (3 sec interval)	Consistent data send every 3 sec	Working

Conclusion

This project successfully demonstrates UART-based communication between two STM32 microcontrollers along with additional functionality including timer-based data transmission, ADC threshold detection, and external interrupt handling. Through periodic UART transmission and reception, the system reliably transfers data and visually indicates communication events using GPIO pins (LEDs). The ADC module effectively monitors environmental light levels via a photoresistor and reacts appropriately based on defined thresholds, sending corresponding signals to the second STM32. Additionally, external interrupt support allows user interaction or sensor-triggered responses to be processed and communicated in real-time. Overall, the project integrates multiple peripherals—USART, TIM, ADC, GPIO, and EXTI—showcasing the STM32's capabilities in embedded system design and real-time control applications.